

SSA Report

05.02.2026

Roberto Di Rosa

Contents

1. Introduction	1
1.1. The Tax-Payer problem	4
2. The Lottery Problem	5
3. The Pattern Problem	5
4. Code	8
4.1. test lottery	8
4.2. test taxpayer	10
4.3. Conclusions	12

1. Introduction

How do we trust smart contracts to be sound? The answer lies in using the [echidna](#), this dynamic testing program allows smart contracts developers to verify the soundness of their code. It works by fuzzing input to the specified contract this will test the invariants specified in such that it wants to break them the more function a contract has the bigger the explorable space will become, this means that testing all possible inputs with all possible combinations is actually quite complex (It's not terminable). The main problem with this tool is that echidna itself is not formally verified therefore even if the tests pass it does not guarantee that the contract is actually sound. So should we trust in echidna and echidna alone, No we should see that the code follows safe patterns, and that the invariants we wrote are actually correct.

```
echidna --test-mode assertion test/testtaxpayer.sol --corpus-dir corpus_dir
```

Listing 1: Echidna cmd example for testing the taxpayer invariants

```
call TimeTest::test_blocks_forward(uint256)(2) <no source map>
├─ call HEVM::roll(4422819) (/home/rob/gits/sec_proj/src/Time.sol:23)
│   └─ 0x
│       └─ 0x
├─ call 0xbd748c717Ca30862991DC44AAf0B4db9F8DE12dA::getLotteryWins() <no source map>
│   └─ (1)
├─ call 0x708Ece5a3D1a6eFBC54e1A134cFF3b47EF346bbE::getLotteryWins() <no source map>
│   └─ (4)
└─ emit AssertionFailed(reason=«The lottery is unfair, Expected val:40 Approx:58 Delta:40») <no source map>
```

Figure 1: Example of echidna showing an AssertionFailed event being triggered for the testing the lottery

The screenshot shows the echidna command-line interface. At the top, there's a list of open files: Lottery.sol, testLottery.sol, State.sol, echidna.yaml, and Taxpayer.sol. Below this, the source code for Lottery.sol is visible, showing a function setTaxAllowance that includes an assertion. The main part of the screenshot is a summary of the test campaign. It shows that 16 tests were run, all of which passed. The campaign was completed successfully, and the user is prompted to press C-c or esc to exit.

Echidna 2.3.1	
Workers: 0/16	Unique instructions: 1647
Seed: 5315931788218589871	Unique codehashes: 2
Calls/s: 11200	Corpus size: 16 seqs
Gas/s: 437484807	New coverage: 9s ago
Total calls: 198880/198880	Slither succeeded
Tests (4) [*]	
AssertionFailed(..): passing	
assertion in invariant_both_married(): passing	
assertion in invariant_is_aged_tax_allowance_good(): passing	
assertion in invariant_is_tax_allowance_good(): passing	
Log (32)	
[2026-02-05 17:45:52.99] [Worker 14] Test limit reached. Stopping.	
[2026-02-05 17:45:52.89] [Worker 15] Test limit reached. Stopping.	
[2026-02-05 17:45:52.87] [Worker 6] Test limit reached. Stopping.	
[2026-02-05 17:45:52.86] [Worker 0] Test limit reached. Stopping.	
[2026-02-05 17:45:52.85] [Worker 12] Test limit reached. Stopping.	
Campaign complete, C-c or esc to exit	

Figure 2: Example of echidna passing for every test in testTaxpayer

The screenshot shows the Echidna test runner interface. At the top, a list of files is open: Lottery.sol, testLottery.sol, State.sol, echidna.yaml, Taxpayer.sol, and testTaxpayer.sol. The main window displays the test results for testLottery.sol. The test configuration is: 1: echidna --config echidna.yaml test/testLottery.sol --test-mode assertion 16. The corpus size is 15. The test results table shows:

Workers: 0/16	Unique instructions: 7101	Chain ID: -
Seed: 2772389834383641526	Unique codehashes: 5	Fetched contracts: 0/0
Calls/s: 1736	Corpus size: 17 seqs	Fetched slots: 0/0
Gas/s: 16326511449	New coverage: 3s ago	
Total calls: 190737/100000	Slither succeeded	

Below the table, the test results are listed:

- AssertionFailed(..): passing
- assertion in players(uint256): passing
- assertion in invariant_test_fairness(): passing
- assertion in lottery_rounds(uint256): passing
- assertion in invariant_test_age(): passing

The log shows 33 entries, all indicating that the test limit was reached and the test was stopped. The campaign is complete, and the user is prompted to press C-c or esc to exit.

Figure 3: Example of echidna passing for every test in testLottery

The screenshot shows the Echidna test runner interface. At the top, a list of files is open: Lottery.sol, testLottery.sol, State.sol, echidna.yaml, Taxpayer.sol, and testTaxpayer.sol. The main window displays the test results for testTaxpayer.sol. The test configuration is: 1: echidna --config echidna.yaml test/testTaxpayer.sol --test-mode assertion 16. The corpus size is 15. The test results table shows:

Workers: 3/16	Unique instructions: 9375	Chain ID: -
Seed: 7569583801813252206	Unique codehashes: 5	Fetched contracts: 0/0
Calls/s: 1529	Corpus size: 38 seqs	Fetched slots: 0/0
Gas/s: 1407075855	New coverage: 24s ago	
Total calls: 41397/100000	Slither succeeded	

Below the table, the test results are listed:

- assertion in invariant_is_tax_allowance_good(): FAILED! with ErrorRevert

The log shows 55 entries, including the failed invariant test. The current action is shrinking the corpus. The campaign is complete, and the user is prompted to press C-c or esc to exit.

Figure 4: Example of echidna not passing for the marry test

```

12  tax_allowance += 2000;
13  pool_tax_allowance += 2000;
14  lottery = address(0);
15  if (spouse != address(0)) {
16      Taxpayer(spouse).setPoolAllowance();
17      if (Taxpayer(spouse).getPoolAllowance() != pool_tax_allowance) {
18          emit AssertionFailed("The spouse has a different pool_tax_allowance");
19      }
20  }
21  }
22
23  function raiseOwnAllowance() public {
24      // require(getYearsSinceBirth() >= oldAge);
25      // require(counter == false);
26      counter = true;
27
28      tax_allowance += 2000;
29      pool_tax_allowance = (pool_tax_allowance + 2000);
30  }
31
32  1: echidna --config echidna.yaml test/testtaxpayer.sol --test-mode assertion 16
33
34  Workers: 2/16      Unique instructions: 9582      Chain ID: -
35  Seed: 5844563261530767604      Unique codehashes: 5      Fetched contracts: 0/0
36  Calls/s: 17175      Corpus size: 37 seqs      Fetched slots: 0/0
37  Gas/s: 14752158093      New coverage: 37s ago
38  Total calls: 875974/1000000      Slither succeeded
39
40  Current action: shrinking 2741/5000 (worker 12)
41  Call sequence:
42  1. Taxpayer.raiseOwnAllowance()
43  2. Taxpayer.marry(0x42997ac9251e5bb8a61f4ff790e5b991ea87fd9b)
44  3. Test.invariant_is_tax_allowance_good()
45
46  Traces:
47  call 0x4f565627233b02eA95bAE7e19F6487Dc05Bb132::getPoolAllowance() (/home/robertodr/gits/sec_proj/test/testtaxpayer.sol:46)
48  ↳ (10000)
49
50  [2026-02-05 17:58:48.61] [Worker 15] Test limit reached. Stopping.
51  [2026-02-05 17:58:46.86] [Worker 6] Test limit reached. Stopping.
52  [2026-02-05 17:58:46.71] [Worker 13] Test limit reached. Stopping.
53  [2026-02-05 17:58:46.51] [Worker 2] Test limit reached. Stopping.
54  [2026-02-05 17:58:46.34] [Worker 8] Test limit reached. Stopping.
55
56  Log (54)
57
58  TERMINAL master $ term:/bin/zsh > Taxpayer f raiseOwnAllowance
59
60

```

Figure 5: Example of echidna not passing because of the age fraud

```

4  // }
5
6  function setTaxAllowance(uint256 ta) public {
7      // require(State(state).isTaxpayerValid(msg.sender));
8      // require(spouse != address(0));
9      // require(msg.sender == spouse);
10     tax_allowance = ta;
11
12     if (Taxpayer(spouse).getTaxAllowance() + ta != (pool_tax_allowance)) {
13         emit AssertionFailed("You and your wife tried to cheat");
14     }
15 }
16
17 function getTaxAllowance() public view returns (uint256) {
18     return tax_allowance;
19 }
20
21 function joinLottery() public {
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

1: echidna --config echidna.yaml test/testtaxpayer.sol --test-mode assertion 16
2
3  Workers: 16/16      Unique instructions: 8941      Chain ID: -
4  Seed: 1396419507065731333      Unique codehashes: 5      Fetched contracts: 0/0
5  Calls/s: 21548      Corpus size: 38 seqs      Fetched slots: 0/0
6  Gas/s: 18908200352      New coverage: 5s ago
7  Total calls: 194837/1000000      Slither succeeded
8
9  assertion in invariant_is_tax_allowance_good(): FAILED! with ErrorRevert
10
11  Current action: shrinking 451/5000 (worker 13)
12  Call sequence:
13  1. Taxpayer.marry(0xf5a2fe45f4f1308502b1c13bb9ef8af136141382)
14  2. Taxpayer.setTaxAllowance(0)
15  3. Test.invariant_is_tax_allowance_good()
16
17  Traces:
18
19  Log (40)
20
21  [2026-02-05 17:54:23.69] [Worker 2] New coverage: 8941 instr, 5 contracts, 38 seqs in corpus
22  [2026-02-05 17:54:22.58] [Worker 1] New coverage: 8941 instr, 5 contracts, 37 seqs in corpus
23  [2026-02-05 17:54:22.23] [Worker 8] New coverage: 8941 instr, 5 contracts, 36 seqs in corpus
24  [2026-02-05 17:54:21.85] [Worker 9] New coverage: 8938 instr, 5 contracts, 35 seqs in corpus
25  [2026-02-05 17:54:21.82] [Worker 7] New coverage: 8938 instr, 5 contracts, 34 seqs in corpus
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

TERMINAL master $ term:/bin/zsh > Taxpayer f setTaxAllowance

```

Figure 6: Example of echidna not passing because of the taxallowance fraud

```

Lottery.sol
1 int256 approx_exp_val = int256(p.getLotteryWins() * (10 ** 18));
2 int256 delta = exp_val - approx_exp_val;
3
4 if (delta < 0) {
5     delta = -delta;
6 }
7 // if (N_OF_ROUNDS > 20){
8 //     emit AssertionFailed("rounds!!!!");
9 // }
10
11 if (delta > 1 * (10 ** 18)) {
12     emit AssertionFailed(string.concat(
13         "The lottery is unfair, Expected val:",
14         Strings.toStringSigned(delta),
15         " Approx:"));
16 }

```

```

1: echidna --config echidna.yaml test/testlottery.sol --test-mode assertion 16

```

Workspaces: 16/16 Seed: 160638314685445718 Calls/s: 1215 Gas/s: 11306857949 Total calls: 9720/100000	Unique instructions: 7638 Unique codehashes: 5 Corpus size: 16 seqs New coverage: 5s ago Slither succeeded	Chain ID: - Fetches contracts: 0/0 Fetches slots: 0/0
---	---	--

```

Tests (5) [*]
assertion in invariant_test_age(): FAILED! with Stop
Call sequence:
1. TestLottery.invariant_test_age()
Traces:
call 0xbd748c717ca30862991dc44aaf8b4db9f80e12da::getYearsSinceBirth() <no source map>
↳ (48)
call 0x708Ece5a301a6efbc54e1a134cFF3b47EF346bbE::getYearsSinceBirth() <no source map>
↳ (48)
call 0xB4FFC0625Fef0541b77925c7A37A55f488bC69d9::getYearsSinceBirth() <no source map>
↳ (86)
Log (25)
[2026-02-05 18:17:20.94] [Worker 15] New coverage: 7638 instr, 5 contracts, 16 seqs in corpus
[2026-02-05 18:17:20.46] [Worker 7] New coverage: 7638 instr, 5 contracts, 15 seqs in corpus
[2026-02-05 18:17:20.35] [Worker 6] New coverage: 7638 instr, 5 contracts, 14 seqs in corpus
[2026-02-05 18:17:20.34] [Worker 0] New coverage: 7638 instr, 5 contracts, 13 seqs in corpus
[2026-02-05 18:17:20.30] [Worker 1] New coverage: 7638 instr, 5 contracts, 12 seqs in corpus
[2026-02-05 18:17:20.24] [Worker 10] New coverage: 7638 instr, 5 contracts, 11 seqs in corpus

```

Figure 7: Example of echidna not passing because of the bad lottery winner Distribution

1.1. The Tax-Payer problem

The first requirement we need to check is if the smart contract written inside of Taxpayer.sol actually enforces sane Taxpayer constraints. We define the correct state using the following invariants:

- **Two way marriage:** If t_1 is married to t_2 , then t_2 must be married to t_1 [cite: 15].
 - $\neg \exists t_1, t_2 : \text{Taxpayer}(t_1) \wedge \text{Taxpayer}(t_2) \wedge$
 $\text{married}(t_1, t_2, \text{True}) \wedge \text{married}(t_2, t_1, \text{False})$
- **A Taxpayer's tax allowance cannot be greater than his pool allowance:** A basic sanity check to ensure individual allowances are subsets of the household pool.
 - $\neg \exists t, p, a : \text{Taxpayer}(t) \wedge \text{get_tax_allowance}(t, a) \wedge$
 $\text{get_pool_tax_allowance}(t, p) \wedge a > p$
- **Married Taxpayers Pooling Consistency:** Married couples must share the exact same pool allowance value. Furthermore, the sum of their individual allowances must equal that shared pool[cite: 38].
 - $\forall t_1, t_2, p_1, p_2, a_1, a_2 : \text{Taxpayer}(t_1) \wedge \text{Taxpayer}(t_2) \wedge$
 $\text{married}(t_1, t_2, \text{True}) \wedge \text{married}(t_2, t_1, \text{True}) \wedge$
 $\text{get_pool}(t_1, p_1) \wedge \text{get_pool}(t_2, p_2) \wedge \text{get_allowance}(t_1, a_1) \wedge \text{get_allowance}(t_2, a_2)$
 $\implies p_1 = p_2 \wedge (a_1 + a_2) = p_1$

- **Allowance Logic Consistency:** A taxpayer's allowance must match their status (Age, Lottery). We define $E(t)$ as the expected allowance for taxpayer t :

$$E(t) = \begin{cases} 9000 & \text{if } t_{\text{is_winner}} \\ 7000 & \text{if } \text{age}(t) \geq 65 \\ 5000 & \text{otherwise} \end{cases}$$

The invariant requires that for any taxpayer t , their stored pool allowance $P(t)$ matches the expected value (summed if married)[cite: 37, 41, 45].

- $\forall t : \text{Taxpayer}(t) \implies \text{get_pool}(t) = (E(t) + (\text{is_married}(t)? E(\text{spouse}(t)) : 0))$

2. The Lottery Problem

For the lottery system (Part 4), we must ensure fairness and correct state transitions. The lottery allows users under 65 to win a higher tax allowance.

- **Lottery Eligibility:** Participants must be valid taxpayers and under the age of 65.
 - $\forall t : \text{Participating}(t) \implies \text{Taxpayer}(t) \wedge \text{age}(t) < 65$
- **Conservation of Winners:** The number of winners must be consistent with the number of completed lotteries. A participant cannot win multiple times arbitrarily without a new lottery round.
- **Unbiased Distribution:** Over a large number of runs N , the frequency of user t winning should be roughly $\frac{1}{|T|}$ where $|T|$ is the participant count.
 - $\text{count_wins}(t) \approx \frac{N}{\text{count_participants}}$

3. The Pattern Problem

Beyond logical invariants, the contract was inspected for structural security vulnerabilities.

- **The age problem:**

First of all storing the age of the customer with the year alone, may not be that bad, the problem is that we need to create a routine, something akin to a cron-job which will call the “haveBirthday()” function once a year, furthermore the function needed some railguards to prevent it from being called more than once each year.

A Taxpayer could call this function until the age would surpass 65 therefore receiving more allowance for his taxes. This is obviously wrong, so we chose to make

the age of the Taxpayer a `unixtimestamp(int256)` this allows us to know exactly when the user has his birthday. We then can calculate the year of the Taxpayer by using the `"getYearsSinceBirth()"` function, this prevents fraudulent behaviour from the users, we then need to account for the raising of the allowance once they reach 65 years. The best thing to do would be to use a [cron-job](#) or something similar.

For this project we chose to do something simpler, the Taxpayer himself is responsible for calling the function `"raiseOwnAllowance()"` which of course follows the CEI pattern, and flags when the user correctly passes the checks, therefore he cannot call it more than once in his life.

- **Check-Effects-Interactions (CEI):** We identified that the original code did not strictly follow the CEI pattern. To prevent potential re-entrancy attacks during external calls (e.g., transfers), we refactored the functions to update state variables **before** interacting with external addresses.

This can be seen in the marriage function, transfer allowance, and many others, in many of these:

1. Some checks were missing
2. Some effects were not happening before the interactions
3. And some interactions were missing.

For example if I marry, my spouse should immediately marry me as well, so the solution is to make another function, `"marry_me()"` that will be called after I set them as my spouse. This and some checks inside of the `marry_me` functions ensure us atomicity.

- **Access Restriction:** We saw how the code lacks even the most basic access restrictions, for example the lottery could be ended by any address inside of the block-chain, by implementing the `onlyBy` modifier we can specify which address can call which functions, in this case the owner.

This problem can also be found in the Taxpayer contract since when we want to marry or when we want to divorce anyone could call this functions, this is solved in the same manner.

- **Method Verification:** Problem, How do we assure that when we marry the address of the spouse we provide actually points to a Legitimate Taxpayer contract?

The first thing that comes to mind is the [ERC165](#) by implementing this we could ask the address which methods it implements, but this is unreliable since we could marry a FAKE Taxpayer contract which is very dangerous.

We found that the more reliable thing to do would be that of actually hashing the code of the contract and comparing the spouse before actually marrying. However since the Taxpayer contract in our implementation actually has immutable values this is the wrong approach. So the solution is to create a Taxpayer Factory which keeps a mapping of each and every taxpayer it has created, so that all the Taxpayer needs to do is to check if the spouse's address is in the mapping. This can be useful for the lottery as well. For simplicity's sake and for the fact that the requirements did not actually tell us if the user could participate in multiple lotteries, we chose to limit the number of simultaneous lotteries shall be one. This can be easily done by creating a State contract which is the factory for the taxpayers and the single lottery, this lottery will be reusable, and we write pre-conditions such that it cannot be called and we also follow the C.E.I. so that we prevent any possible re-entrant attack.

4. Code

4.1. test lottery

```

function proxy_player_commit(uint256 playerIndex) internal {
    uint256 idx = playerIndex % NUM_PLAYERS;
    players[idx].joinLottery();
}

function player_round(uint256 idx) internal {
    proxy_player_commit(idx);
}

function lottery_rounds(uint256 _seed) public {
    N_OF_ROUNDS+=1;
    s.proxy_startlottery();

    for (uint256 index = 0; index < players.length; index++) {
        player_round(index);
    }

    t.test_vesting(1 days); // NOTE: This makes time pass by one day
    s.proxy_endlottery(_seed);

    t.test_blocks_forward(2); //NOTE: Makes the blocks go forward by one
}

function safe_exp_value(uint256 player_idx) internal {
    Taxpayer p = players[player_idx];
    if (p.getYearsSinceBirth() ≥ 65) {
        return;
    }
    int256 exp_val = int256((N_OF_ROUNDS) * (( 10 ** 18 ) / get_len()));
    int256 approx_exp_val = int256(p.getLotteryWins() * ( 10 ** 18 ));
    int256 delta = exp_val - approx_exp_val;

    if (delta < 0) {
        delta = -delta;
    }

    if (delta > 10 * ( 10 ** 18 )) {
        emit AssertionFailed(string.concat(
            "The lottery is unfair, Expected val:",
            Strings.toStringSigned(delta),
            " Approx:",
            Strings.toStringSigned(approx_exp_val),
            " Delta:",
            Strings.toStringSigned(delta)
        ));
    }
}

function invariant_test_fairness() public {
    for (uint256 index = 0; index < players.length; index++) {
        safe_exp_value(index);
    }
}

```

```
function invariant_test_age() public {
  for (uint256 index = 0; index < players.length; index++) {

    bool i = lot.getTaxPayer(address( players[index] ));

    if(i && ( players[index].getYearsSinceBirth() ≥ 65 )){
      emit AssertionFailed("Old man in lottery");
    }
  }
}
```

4.2. test taxpayer

```
function forEach(function(Taxpayer) internal constraint) internal {
  for (uint256 index = 0; index < N_OF_TAXPAYER; index++) {
    constraint(taxpayer[index]);
  }
}
```

Listing 4: $\forall$ utility.

```
function checkMarried(Taxpayer t1) internal {
  Taxpayer spouse = t1.get_spouse();
  if (address(spouse) == address(0)) return;

  if (address(spouse.get_spouse()) ≠ address(t1)) {
    emit AssertionFailed("The spouse is different");
  }
}

function invariant_are_both_married() public {
  forEach(checkMarried);
}
```

Listing 5: Check Married Invariant this checks the $\text{me_married_with_spouse} \iff \text{spouse_married_with_me}$ constraint.

```

function checkAllowance(Taxpayer t1) internal {
    if (t1.getTaxAllowance() > t1.getPoolAllowance()) {
        emit AssertionFailed("Too much money saved in taxes");
    }
    Taxpayer sp = t1.get_spouse();

    if (address(sp) != address(0)) {
        if (t1.getPoolAllowance() != sp.getPoolAllowance()) {
            emit AssertionFailed("The pool allowance must be equal for both spouses");
        }
        if ((t1.getTaxAllowance() + sp.getTaxAllowance()) != t1.getPoolAllowance()) {
            emit AssertionFailed("Too much money saved in taxes naughty couple");
        }
    }
}

function invariant_is_tax_allowance_good() public {
    forEach(checkAllowance);
}

```

Listing 6:

```

function checkAgeAllowance(Taxpayer t1) internal {
    uint256 my_mod = 0;
    uint256 my_num = 5000;
    if (t1.getYearsSinceBirth() ≥ oldAge && t1.get_counter()) {
        my_mod += 2000;
    }
    my_mod += (t1.getLotteryWins() * 2000);
    Taxpayer sp = t1.get_spouse();
    if (address(sp) != address(0)) {
        my_num = my_num * 2;
        if (sp.getYearsSinceBirth() ≥ oldAge && sp.get_counter()) {
            my_mod += 2000;
        }

        my_mod += (sp.getLotteryWins() * 2000);
    }
    if (t1.getPoolAllowance() != (my_num + my_mod)) {
        emit AssertionFailed(string.concat("The poolTaxAllowance is wrong",
Strings.toString(my_num + my_mod)));
    }
}

function invariant_is_aged_tax_allowance_good() public {
    forEach(checkAgeAllowance);
}

```

Listing 7: This checks the allowance counting the age and the lottery wins.

4.3. Conclusions

Using Echidna, we successfully verified the core invariants of the `Taxpayer` system. The initial fuzzing campaign revealed violations in the marriage logic (one-way marriage bugs) and tax pooling calculations, which were resolved by enforcing the bidirectional constraints described in Section 2. While for the lottery we managed to re-write the code base making it impossible to instantiate the lottery more than once and making a person only win once and not more times at the same time, while having good enough player winning distribution.